21.4 开发自定义扩展

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~850k tokens，输出 ~80k tokens（本章合计）

前三节我们从架构、API 和实例三个层面理解了 OpenClaw 的扩展机制。本节进入实践环节——手把手指导如何开发一个自定义通道扩展，让读者具备独立开发新通道的能力。

21.4.1 扩展脚手架搭建

目录结构

一个最小可用的通道扩展需要以下文件：

```
my-channel/
├── package.json              # npm 包配置
├── openclaw.plugin.json      # 插件清单（必须）
├── index.ts                  # 入口文件
├── tsconfig.json             # TypeScript 配置（可选但推荐）
└── src/
    ├── channel.ts            # ChannelPlugin 定义
    ├── monitor.ts            # 消息监听逻辑
    ├── send.ts               # 消息发送逻辑
    └── runtime.ts            # 运行时引用缓存
```

package.json

```json
{
  "name": "openclaw-channel-mychat",
  "version": "0.1.0",
  "type": "module",
  "openclaw": {
    "extensions": ["index.ts"]
  },
  "dependencies": {
    "mychat-sdk": "^1.0.0"
  }
}
```

关键配置：

字段	必填	说明
`openclaw.extensions`	✅	声明入口文件路径数组，被插件发现机制读取
`type: "module"`	推荐	ESM 模块格式
`dependencies`	按需	第三方 SDK 依赖

openclaw.plugin.json

```json
{
  "id": "mychat",
  "channels": ["mychat"],
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": {
        "type": "string",
        "description": "MyChat Bot API Key"
      },
      "webhookPort": {
        "type": "number",
        "description": "Webhook listening port",
        "default": 4000
      }
    },
    "required": ["apiKey"]
  }
}
```

清单字段说明：

字段	用途
`id`	全局唯一标识符，用于 `plugins.entries.mychat` 配置
`channels`	声明此插件提供的通道 ID，用于 Gateway 自动加载
`configSchema`	JSON Schema，插件加载时验证 `plugins.entries.mychat.config`

runtime.ts — 运行时缓存

```
// src/runtime.ts
import type { PluginRuntime } from "openclaw/plugin-sdk";

let runtime: PluginRuntime | null = null;

export function setMyChatRuntime(rt: PluginRuntime) {
  runtime = rt;
}

export function getMyChatRuntime(): PluginRuntime {
  if (!runtime) throw new Error("MyChat runtime not initialized");
  return runtime;
}
```

这个模式在 21.3 节中反复出现。由于 `PluginRuntime` 只在 `register()` 调用时注入，其他模块无法在导入时获得它，因此需要一个模块级缓存。

index.ts — 入口文件

```
// index.ts
import type { OpenClawPluginApi } from "openclaw/plugin-sdk";
import { emptyPluginConfigSchema } from "openclaw/plugin-sdk";
import { myChatPlugin } from "./src/channel.js";
import { setMyChatRuntime } from "./src/runtime.js";

const plugin = {
  id: "mychat",
  name: "MyChat",
  description: "MyChat channel plugin",
  configSchema: emptyPluginConfigSchema(),
  register(api: OpenClawPluginApi) {
    setMyChatRuntime(api.runtime);
    api.registerChannel({ plugin: myChatPlugin });
  },
};

export default plugin;
```

> **注意**：如果你的插件有自定义配置（不是 empty schema），可以将 `emptyPluginConfigSchema()` 替换为自定义的 Zod Schema 或手写的 `safeParse` 函数。 `emptyPluginConfigSchema()` 的实现只接受空对象或 `undefined` 。

21.4.2 实现通道适配器接口

最小可用的 ChannelPlugin

`ChannelPlugin` 有近 20 个可选适配器槽位，但**大多数是可选的**。一个最小可用的通道只需要实现 `id` 、 `meta` 、 `capabilities` 和 `config` ：

```typescript
// src/channel.ts
import type { ChannelPlugin, OpenClawConfig } from "openclaw/plugin-sdk";
import { DEFAULT_ACCOUNT_ID } from "openclaw/plugin-sdk";

type MyChatAccount = {
  accountId: string;
  enabled: boolean;
  configured: boolean;
};

const meta = {
  id: "mychat",
  label: "MyChat",
  selectionLabel: "MyChat Messenger",
  blurb: "MyChat 即时通讯平台",
  order: 100,    // 在通道选择列表中的排序权重
};

export const myChatPlugin: ChannelPlugin<MyChatAccount> = {
  id: "mychat",
  meta,

  // 能力声明——告诉 OpenClaw 这个通道支持什么
  capabilities: {
    chatTypes: ["direct", "group"],   // 支持私聊和群聊
    polls: false,                     // 不支持投票
    reactions: false,                 // 不支持表情回复
    threads: false,                   // 不支持线程
    media: true,                      // 支持媒体（图片/文件）
  },

  // 配置适配器——账号管理
  config: {
    listAccountIds: () => [DEFAULT_ACCOUNT_ID],

    resolveAccount: (cfg) => ({
      accountId: DEFAULT_ACCOUNT_ID,
      enabled: cfg.channels?.mychat?.enabled !== false,
      configured: Boolean(cfg.channels?.mychat?.apiKey),
    }),

    defaultAccountId: () => DEFAULT_ACCOUNT_ID,
```

```
  isConfigured: (account) => account.configured,

  describeAccount: (account) => ({
    accountId: account.accountId,
    enabled: account.enabled,
    configured: account.configured,
  }),

  resolveAllowFrom: ({ cfg }) =>
    cfg.channels?.mychat?.allowFrom ?? [],

  formatAllowFrom: ({ allowFrom }) =>
    allowFrom.map((entry) => String(entry).trim().toLowerCase()).filter(Boolean),

  setAccountEnabled: ({ cfg, enabled }) => ({
    ...cfg,
    channels: {
      ...cfg.channels,
      mychat: { ...cfg.channels?.mychat, enabled },
    },
  }),

  deleteAccount: ({ cfg }) => {
    const next = { ...cfg } as OpenClawConfig;
    const nextChannels = { ...cfg.channels };
    delete nextChannels.mychat;
    next.channels = nextChannels;
    return next;
  },
},

// Gateway 适配器——消息监听
gateway: {
  startAccount: async (ctx) => {
    const { startMyChatMonitor } = await import("./monitor.js");
    return startMyChatMonitor({
      cfg: ctx.cfg,
      abortSignal: ctx.abortSignal,
      runtime: ctx.runtime,
      log: ctx.log,
    });
  },
```

```
  },
};
```

能力声明的意义

`capabilities` 不仅仅是元数据，OpenClaw 的其他模块会根据它做决策：

```
capabilities: {
  chatTypes: ["direct", "group"],  // → Agent 工具的 target 参数会包含群组选项
  polls: false,                    // → Agent 不会生成投票相关的工具调用
  threads: false,                  // → 消息分发不会尝试维护线程上下文
  media: true,                     // → Agent 可以发送图片和文件
}
```

逐步扩展适配器

随着功能完善，你可以逐步添加更多适配器：

第 1 阶段（最小可用）： `config` + `gateway`

第 2 阶段（安全控制）：添加 `security` + `pairing`

```
security: {
  resolveDmPolicy: ({ account }) => ({
    policy: "pairing",               // 新联系人需要审批
    allowFrom: account.allowFrom,
    policyPath: "channels.mychat.dm.policy",
    allowFromPath: "channels.mychat.dm.allowFrom",
  }),
},

pairing: {
  idLabel: "myChatUserId",
  normalizeAllowEntry: (entry) => entry.replace(/^mychat:/i, ""),
  notifyApproval: async ({ cfg, id }) => {
    await sendMyChat(cfg, id, "已通过验证，可以开始对话了！");
  },
},
```

第 3 阶段（消息能力增强）：添加 `outbound` + `messaging` + `actions`

第 4 阶段（运维功能）：添加 `status` + `directory` + `onboarding`

21.4.3 注册到插件系统

安装方式

有三种方式将自定义扩展注册到 OpenClaw：

方式一：工作区级安装（推荐开发调试）

将扩展目录放到项目的 `.openclaw/extensions/` 下：

```
# 假设 OpenClaw 工作区在 ~/openclaw-workspace
mkdir -p ~/openclaw-workspace/.openclaw/extensions
cp -r my-channel ~/openclaw-workspace/.openclaw/extensions/
```

OpenClaw 启动时会自动发现并加载。无需任何配置。

方式二：全局安装

放到用户级配置目录：

```
cp -r my-channel ~/.config/openclaw/extensions/
```

所有工作区都能使用此扩展。

方式三：配置路径安装

在 `openclaw.config.yaml` 中显式指定路径：

```
plugins:
  loadPaths:
    - /path/to/my-channel
```

适合开发时指向源码目录，修改后不需要复制。

配置启用

在 `openclaw.config.yaml` 中添加扩展配置：

```yaml
plugins:
  entries:
    mychat:
      enabled: true
      config:
        apiKey: "your-api-key-here"
        webhookPort: 4000

channels:
  mychat:
    enabled: true
    allowFrom:
      - "user123"
      - "user456"
```

验证加载

启动 OpenClaw 后，可以通过 Gateway 日志确认插件是否加载成功：

```
[plugins] loaded mychat (bundled) — 1 channel, 0 tools, 0 hooks
```

如果加载失败，日志中会出现诊断信息：

```
[plugins] error: mychat — config validation failed: apiKey is required
```

也可以通过 Gateway RPC 查看已加载的插件列表：

```
openclaw status
# 输出中应包含 mychat 通道的状态信息
```

21.4.4 测试与调试

单元测试

扩展可以独立进行单元测试。推荐使用 Vitest（OpenClaw 官方扩展的测试框架）：

```
// src/channel.test.ts
import { describe, it, expect } from "vitest";
import { myChatPlugin } from "./channel.js";

describe("myChatPlugin", () => {
  it("should have correct id and meta", () => {
    expect(myChatPlugin.id).toBe("mychat");
    expect(myChatPlugin.meta.label).toBe("MyChat");
  });

  it("should resolve account correctly", () => {
    const cfg = {
      channels: {
        mychat: { enabled: true, apiKey: "test-key" },
      },
    };
    const account = myChatPlugin.config.resolveAccount(cfg as any);
    expect(account.enabled).toBe(true);
    expect(account.configured).toBe(true);
  });

  it("should detect unconfigured account", () => {
    const cfg = { channels: {} };
    const account = myChatPlugin.config.resolveAccount(cfg as any);
    expect(account.configured).toBe(false);
  });
});
```

OpenClaw 官方扩展中的测试可以作为参考——例如 `extensions/msteams/src/` 包含 `attachments.test.ts` 、 `inbound.test.ts` 、 `messenger.test.ts` 等多个测试文件。

集成测试

对于需要验证消息收发完整链路的场景，可以通过 Mock 消息的方式进行集成测试：

```typescript
// src/monitor.test.ts
import { describe, it, expect, vi } from "vitest";

describe("monitorMyChatProvider", () => {
  it("should process incoming message", async () => {
    const dispatchMock = vi.fn();

    // 模拟收到一条消息
    const message = {
      from: "user123",
      text: "你好",
      timestamp: Date.now(),
    };

    await processIncoming(message, { dispatch: dispatchMock });

    expect(dispatchMock).toHaveBeenCalledWith(
      expect.objectContaining({
        From: "user123",
        Body: "你好",
      })
    );
  });
});
```

调试技巧

1. 启用详细日志

在配置中开启 verbose 模式：

```yaml
logging:
  level: debug
```

或在启动时设置环境变量：

```bash
OPENCLAW_LOG_LEVEL=debug openclaw start
```

2. 使用 Plugin Runtime 的日志器

```
// 在 register() 中
register(api: OpenClawPluginApi) {
  const log = api.logger;
  log.info("MyChat plugin loading...");
  log.debug("Plugin config: " + JSON.stringify(api.pluginConfig));
}

// 在其他模块中通过 runtime
const runtime = getMyChatRuntime();
const log = runtime.logging.getChildLogger({ name: "mychat" });
log.info("Processing message...");
```

3. 诊断事件（Diagnostic Events）

如果你需要监控扩展的运行状态，可以使用诊断事件系统：

```
import { emitDiagnosticEvent, isDiagnosticsEnabled } from "openclaw/plugin-sdk";

if (isDiagnosticsEnabled()) {
  emitDiagnosticEvent({
    type: "webhook_received",
    channel: "mychat",
    timestamp: Date.now(),
    metadata: { messageId: msg.id },
  });
}
```

4. 消息处理链路调试

对于入站消息处理，关键的调试检查点：

```
Webhook 接收 → 允许列表检查 → 消息规范化 → Agent 路由 → Agent 处理 → 回复分发 → 发送
```

如果消息"消失"了，最常见的原因是：

- 发送者不在 `allowFrom` 列表中
- 消息格式不符合预期，规范化失败
- Agent 路由未匹配到任何 Agent
- 配置中 `enabled: false`

发布到 npm

完成开发后，可以将扩展发布为 npm 包：

```
# 添加必要字段
npm version patch
npm publish --access public
```

其他用户安装：

```
npm install -g openclaw-channel-mychat
# 或在工作区安装
npm install openclaw-channel-mychat
```

然后在 `openclaw.config.yaml` 中添加 loadPaths 指向安装目录，或将其软链到 `~/.config/openclaw/extensions/` 。

本节小结

1. **最小可用扩展只需 5 个文件**：`package.json` 、 `openclaw.plugin.json` 、 `index.ts` 、 `src/channel.ts` 、 `src/runtime.ts` 。其中 `openclaw.plugin.json` 清单和入口文件的 `register()` 函数是必须的。
2. **ChannelPlugin 的 20 个适配器槽位大多是可选的**，推荐采用渐进式开发：先实现 `config` + `gateway` （最小可用），再逐步添加 `security` 、 `outbound` 、 `status` 等。
3. **三种安装方式**适合不同场景：工作区安装用于开发调试，全局安装用于个人使用，配置路径安装用于源码开发。
4. **测试应分两层**：单元测试验证 ChannelPlugin 的各个适配器实现，集成测试验证消息收发完整链路。
5. **调试的核心是日志和诊断事件**：通过 `api.logger` 、 `runtime.logging.getChildLogger()` 和 `emitDiagnosticEvent()` 三个层次的日志，可以定位消息处理链路中的任何问题。